Requirements File

Requirements File

You can simplify the installation of many packages by using a requirements file. The `requirements.txt` file is a text file that lists the packages and versions used in a project. Python can then use this list to install the specific version of each package listed.

Before we can install from this list, we need to first create the `requirements.txt` file. Make sure that you are in the directory with your virtual environment. Then activate the environment. We want to generate the requirements only for the environment, not your general Python installation.

```
cd example1
source example-venv/bin/activate
```

Use the redirect operator (`>`) to take the output from `freeze` and write it to the `requirements.txt` file. This process is also called pinning as the exact version will be recorded in the requirements file. After running the command, click on the link below to open the requirements file.

```
python3 -m pip freeze > requirements.txt
```

Open requirements.txt

Installing from a Requirements File

Another benefit of the requirements file is that you can use it as a list for packages to install. Let's start by creating a second virtual environment. We first need to deactivate the virtual environment and exit the current directory, `example1` . Then we are going to make a directory called `example2` and change to this directory.

```
deactivate
cd ..
mkdir example2
cd example2
```

Now, create a new virtual environment (we aren't going to use a special name, `venv` is sufficient). Then activate the new virtual environment. Your prompt should have `(venv)` at the beginning.

```
python3 -m venv venv
source venv/bin/activate
```

We are ready to use the requirements file in the `example1` directory. Give `pip` the `-r` flag to tell Python that the requirements are found in the file `requirements.txt` .

```
python3 -m pip install -r ../example1/requirements.txt
```

▶ **Did you notice?**

Now list the packages in the `example2` virtual environment. You should see the same ones as in `example1` .

```
python3 -m pip list
```

Specifying Package Versions

Looking again at the contents of the `requirements.txt` file, you will notice that the `==` operator is used with the version number for each package. This tells Python to install exactly this version.

Open requirements.txt

You may find yourself wanting to keep the same packages but upgrade to the latest version. We can do this by making some changes to the requirements file. Replace all of the `==` operators with `>=` . This tells Python to use this or a later version. **Note**, your list of packages and version numbers may be different from the time of writing.

```
beautifulsoup4==4.11.1
numpy==1.19.5
pandas==1.1.5
pkg_resources==0.0.0
python-dateutil==2.8.2
pytz==2022.2.1
scipy==1.5.4
six==1.16.0
soupsieve==2.3.2.post1
```

After you made these changes, you can tell Python to upgrade the packages in the requirements file by using the `-U` flag.

```
python3 -m pip install -U -r ../example1/requirements.txt
```

One final use case for the requirements file is separating production from development environments. You are **not** going to create anything, but it is worth understanding how this works. Some developers use testing packages not found in the standard library. For example, `pytest` is popular for testing, but it is not needed for production. You would create your `requirements.txt` file without any testing packages. Then you would make another version called `requirements_dev.txt` . The contents of this file should look like this:

```
# requirements_dev.txt

-r requirements.txt
pytest>=7.1.1
```

Installing from the `requirements_dev.txt` file will first install all of the packages from `requirements.txt` . Then it will install the `pytest` package. When it comes time to deploy your package, you can install the needed packages from `requirements.txt` which excludes any testing packages.

Reading Question

Select all of the things you can do with a requirements file. **Hint**, there is more than one correct answer.

- [ ] Specify the exact package version to use
- [ ] Specify a package version greater than or equal to a base version
- [ ] Upgrade installed packages
- [ ] Install multiple packages at once

All of these are things that you can do with a `requirements.txt` file. You can install all of the packages with the `-r` flag. You can upgrade the packages with the `-U` flag. Using the `==` operator specifies the exact version to use. And using the `>=` operator means you can use any version greater than or equal to the stated version in the file.

Check It!

Next